

PRÁCTICA 3

Cronómetro con precisión de centésimas

DOCUMENTO	ASIGNATURA	TITULACIÓN	CENTRO
Del alumnado	**Arq. de Computadores II**	**Grado en Ing. Informática · 2.º**	**ESEI · Universidade de Vigo**

Contenido

1. Ficha de la práctica 3

2. Objetivos de aprendizaje 3

3. El problema 4

4. Modelo del sistema 4

4.1 La base de tiempo 4

4.2 Cómo se guarda el tiempo 5

4.3 Qué se ve en el display 5

5. Comportamiento exigido 6

6. Desarrollo paso a paso 6

Paso 1. Mostrar un valor fijo 6

Paso 2. La base de tiempo 6

Paso 3. Los acarreos 7

Paso 4. Refrescar solo cuando cambia 7

Paso 5. Mide tu error 7

Paso 6. Caza el acarreo con el depurador 8

6.1 Ampliación opcional: el cronómetro de la bomba 8

7. Criterios de aceptación 10

8. Criterios de evaluación 10

9. Entregables 11

10. Uso de asistentes de inteligencia artificial 12

Anexo A. La función HAL_GetTick() 13

Anexo B. Por qué el error es exactamente del 9,1 % 13

Anexo C. Conexionado 14

1. Ficha de la práctica

CAMPO	VALOR
Sesión	Sesión 4 de 13
Duración	2 horas presenciales
Modalidad	Individual, en el laboratorio
Requisitos previos	P2: bucle no bloqueante y biblioteca TM1638
Material	Placa NUCLEO-F401RE · módulo LED&KEY (TM1638) · STM32CubeIDE · biblioteca TM1638
Entregables	Proyecto CubeIDE, medida del error y memoria breve (ver apartado 9)
Plazo	Una semana desde el final de la sesión, por Moovi

2. Objetivos de aprendizaje

Al terminar esta práctica serás capaz de:

- Construir una **base de tiempo periódica** con `HAL_GetTick()` que no acumule deriva.
- Explicar por qué > y >= no son intercambiables cuando se comparan tiempos, y cuánto cuesta equivocarse.
- Descomponer una magnitud en cifras y **propagar el acarreo** entre unidades.
- Refrescar una pantalla **solo cuando el dato cambia**, en lugar de en cada vuelta del bucle.
- Medir el error de tu propio reloj contra una referencia externa y expresarlo en porcentaje.

> Esta práctica parece trivial y no lo es. Un cronómetro son veinte líneas, pero dos caracteres mal puestos hacen que **atrase casi seis segundos por minuto**. Y lo peor: funciona, se ve bonito y nadie lo nota hasta que lo compara con un reloj de verdad.

3. El problema

Medir tiempo en un microcontrolador parece fácil porque hay una función que da la hora: `HAL_GetTick()` devuelve los milisegundos transcurridos desde el arranque. Lo difícil no es leerla, es **usarla para marcar un ritmo** sin que el ritmo se vaya desviando.

En P2 ya construiste una base de tiempo de 100 ms para mover una puerta. Allí una desviación pequeña no se notaba: la puerta tardaba un poco más y ya está. Aquí sí se nota, porque lo que estás construyendo **es** el instrumento de medida.

4. Modelo del sistema

4.1 La base de tiempo

MAGNITUD	VALOR	CONSTANTE EN EL CÓDIGO
Resolución de `HAL_GetTick()`	1 ms	—
Periodo de cuenta	10 ms = 1 centésima	`TICK_MS`
Alcance del cronómetro	000,00 a 999,99 s	—
Al llegar al máximo	vuelve a 000,00	—

4.2 Cómo se guarda el tiempo

El cronómetro se lleva **descompuesto en cifras**, en un array de cuatro posiciones, y no como un entero de milisegundos. No es la forma más eficiente, y es deliberado: el objeto de la práctica es manejar el desbordamiento de cada unidad, que es lo que ocurre en cualquier contador real.

ÍNDICE	QUÉ GUARDA	RANGO
`c[0]`	Centésimas	0 a 99
`c[1]`	Unidades de segundo	0 a 9
`c[2]`	Decenas de segundo	0 a 9
`c[3]`	Centenas de segundo	0 a 9

> Ojo con `c[0]`: cuenta **centésimas de 0 a 99**, no décimas. Al desbordar incrementa directamente las unidades de segundo, porque 100 centésimas son un segundo. Confundir centésimas con décimas es el origen más frecuente de acarreos mal planteados.

4.3 Qué se ve en el display

Seis caracteres, en el formato `SSS.CC`. El punto no ocupa posición propia: la biblioteca TM1638 lo une al dígito anterior, así que en pantalla se ven seis dígitos con el punto entre el tercero y el cuarto.

EJEMPLOS DE LO QUE DEBE VERSE

```
000.00     al arrancar
009.99     nueve segundos y noventa y nueve centesimas
010.00     la centesima siguiente
999.99     el maximo, justo antes de volver a cero
```

5. Comportamiento exigido

1. Al arrancar, el cronómetro muestra **000.00** y empieza a contar de inmediato.
2. Cada **10 ms** avanza una centésima.
3. Los acarreos se propagan unidad por unidad: 100 centésimas hacen un segundo, 10 segundos una decena, 10 decenas una centena.
4. Al pasar de **999.99** vuelve a **000.00** sin saltos ni valores intermedios raros.
5. El display se actualiza **solo cuando el valor cambia**, no en cada vuelta del bucle.
6. El bucle principal **no contiene ninguna espera bloqueante**. Ni `HAL_Delay`, ni bucles de espera activa.

6. Desarrollo paso a paso

Paso 1. Mostrar un valor fijo

Antes de contar nada, consigue pintar `000.00` en el display. Escribe una función que reciba el array de cifras y componga la cadena de seis caracteres. Pruébala con valores inventados: 0, 123 y 999 con sus centésimas.

> Escribe esa función **fuera de `main`**, como función de fichero. Definir funciones dentro de otra función es una extensión de GNU: compila con este compilador y no compila con otros. Es el mismo aviso que en P2.

Paso 2. La base de tiempo

Ahora haz que avance una centésima cada 10 ms. Este es el corazón de la práctica y donde están las dos trampas:

BASE DE TIEMPO CORRECTA

```
static uint32_t tCuenta = 0;

tCuenta = HAL_GetTick();          /* una vez, antes del bucle */
```

```
if (HAL_GetTick() - tCuenta >= TICK_MS) {
    tCuenta += TICK_MS;
    /* avanzar una centesima */
}
```

Trampa 1: >= y no >. Con > la condición no se cumple hasta los 11 ms, así que cada centésima dura un 10 % de más. El cronómetro atrasa **5,5 segundos por minuto**. Se ve perfecto y mide mal.

Trampa 2: sumar el periodo, no reasignar. `tCuenta += TICK_MS` conserva el desfase con el que se llegó a comprobar la condición; `tCuenta = HAL_GetTick()` lo descarta, y ese error se acumula vuelta tras vuelta. Cuanto más tarde el bucle en dar una pasada, peor: con un bucle de 3 ms el atraso llega a 10 s por minuto.

Paso 3. Los acarreos

Cada vez que avanza una centésima hay que propagar el acarreo hacia arriba. Cuatro comprobaciones encadenadas, en orden, de la unidad más pequeña a la más grande.

Compruébalo en los puntos difíciles, que son siempre los cambios de decena: 0,99 → 1,00; 9,99 → 10,00; 99,99 → 100,00; y 999,99 → 000,00.

Paso 4. Refrescar solo cuando cambia

Si escribes en el display en cada vuelta del bucle, lo estarás redibujando miles de veces por segundo. El TM1638 se gobierna por un puerto serie lento, así que eso consume tiempo para nada y además **empeora la medida**, porque alarga la vuelta del bucle.

Mueve la actualización del display dentro del bloque que se ejecuta cada 10 ms.

Paso 5. Mide tu error

Arranca el cronómetro a la vez que el de tu móvil y déjalos correr **cinco minutos**. Anota la diferencia y exprésala en porcentaje. Ese número es el entregable del apartado 9.

> Si te sale un error en torno al **9 %**, casi seguro que tienes la trampa 1. Si te sale pequeño pero crece con el tiempo, es la trampa 2. Si te sale por debajo del 0,1 %, lo has hecho bien.

Paso 6. Caza el acarreo con el depurador

Los acarreos son el sitio donde más fallan los cronómetros, y esperar a que ocurran mirando la pantalla es lento. Se pueden provocar a voluntad.

7. Pon un **punto de ruptura condicional** dentro de `CronometroAvanza`, con la condición `c[0] == 99`. Ojo al nombre: dentro de la función el array llega como `c`, aunque fuera se llame `cronometro`.
8. Deja correr. El programa se detendrá justo antes del acarreo.
9. Avanza **paso a paso** y mira cómo se propaga: centésimas a cero, unidades más uno.

> Esto convierte una espera de hasta un segundo en algo instantáneo, y permite comprobar los cuatro acarreos —centésimas, unidades, decenas y la vuelta a cero— sin esperar mil segundos al último.

Aprovecha para observar un efecto del capítulo 11: al reanudar después de haber estado parado un rato, **el cronómetro no salta**. Explica en la memoria por qué, sabiendo cómo se construye la base de tiempo.

6.1 Ampliación opcional: el cronómetro de la bomba

Para quien termine pronto o quiera trabajarlo en casa. Sobre el cronómetro anterior:

- Cuenta **atrás** desde 10,00 s en lugar de hacia delante.
- Permite introducir un código de 4 bits con los pulsadores S1 a S4 del módulo.
- El flanco de S8 confirma el código.
- Cuatro intentos como máximo, indicados con los LEDs que quedan encendidos.
- Si se agota el tiempo o los intentos, los ocho LEDs parpadean sin parar.

No entra en la evaluación de esta práctica, pero es un buen entrenamiento para el proyecto final.

7. Criterios de aceptación

Nº	PRUEBA	RESULTADO EXIGIDO
CA-1	Encender la placa y observar el display.	Muestra **000.00** y empieza a contar de inmediato.
CA-2	Cronometrar 60 s con un reloj externo y comparar.	La diferencia es **menor de 0,5 s**, es decir un error por debajo del 1 %.
CA-3	Cronometrar 5 minutos con un reloj externo.	La diferencia **no crece de forma apreciable** respecto a la de un minuto. Si crece, hay deriva.
CA-4	Observar el paso de 9,99 a 10,00 y de 99,99 a 100,00.	Los acarreos se propagan bien, sin dígitos intermedios erróneos.
CA-5	Dejarlo correr más de 1000 s.	Al pasar de **999.99** vuelve a **000.00** y sigue contando con normalidad.
CA-6	Revisar el código entregado.	No hay `HAL_Delay` ni esperas activas en el bucle, y las funciones auxiliares están **fuera de `main`**.

8. Criterios de evaluación

> **Condición previa.** El proyecto debe compilar en STM32CubeIDE **sin errores y sin avisos**. Un proyecto que no compila no se evalúa: se devuelve para corrección y se califica sobre 5.

La práctica se califica sobre 10 puntos. Cada criterio se valora en tres niveles: no alcanzado (0 puntos), alcanzado a medias (mitad del peso) y alcanzado (peso completo).

CRITERIO	QUÉ SE VALORA	ALCANZADO A MEDIAS	PESO
1. Precisión	CA-2: el error en un minuto es menor del 1 %.	Cuenta, pero con un error de entre el 1 % y el 10 %.	2,5
2. Ausencia de deriva	CA-3: el error no crece con el tiempo.	El error en cinco minutos es más de tres veces el de un minuto.	2,0
3. Acarreos	CA-4 y CA-5: propagación correcta y vuelta a cero.	Falla solo en la vuelta a cero, o solo en un cambio de decena.	2,0
4. Código no bloqueante	CA-6: sin `HAL_Delay` ni esperas activas.	Sin `HAL_Delay`, pero el display se refresca en cada vuelta del bucle.	1,5
5. Calidad del código	Funciones fuera de `main`, constantes con nombre, sin avisos.	Funciona pero quedan números sueltos o alguna función anidada.	1,0
6. Medida del error	Paso 5: dato experimental propio, expresado en porcentaje.	Aporta la medida pero sin calcular el porcentaje, o sin comparar el minuto con los cinco.	1,0

9. Entregables

- **Proyecto de STM32CubeIDE** comprimido, **sin las carpetas Debug ni Release**. Nombre: ACII_P3_Apellido1_Apellido2_Nombre.zip
- **Medida del error**: la diferencia observada en un minuto y en cinco minutos, y el porcentaje correspondiente.

- **Memoria breve**, máximo dos páginas, que responda a estas dos preguntas:
    - Qué le pasa al cronómetro si se escribe > en lugar de >=, y por qué el error resulta ser justo del 9,1 %.
    - Por qué `tCuenta += TICK_MS` no acumula deriva y `tCuenta = HAL_GetTick()` sí, y en qué situación se nota más la diferencia.

Entrega por Moovi en el plazo de la ficha. La práctica se defiende en el laboratorio: se te pedirá ejecutar los criterios de aceptación sobre tu propia placa y explicar cualquier parte del código.

10. Uso de asistentes de inteligencia artificial

Se permite el uso de asistentes de IA como herramienta de consulta y de depuración, con tres condiciones:

- **Declararlo** en la memoria: qué asistente y para qué lo has usado.
- **Entender lo que entregas.** En la defensa se te puede pedir que expliques cualquier línea o que la modifiques en el momento.
- **No sustituir el trabajo de medida.** El error de tu cronómetro lo mides tú, con un reloj y un papel.

Anexo A. La función HAL_GetTick()

Devuelve el número de milisegundos transcurridos desde que arrancó el sistema. La mantiene el temporizador SysTick, que la biblioteca HAL configura por su cuenta para interrumpir cada milisegundo.

CARACTERÍSTICA	VALOR
Tipo devuelto	`uint32_t`
Resolución	1 milisegundo
Desbordamiento	A los 2^{32} ms, es decir unos **49,7 días**

> La resta `HAL_GetTick() - tCuenta` **funciona correctamente aunque el contador desborde**, siempre que las dos variables sean `uint32_t`. La aritmética sin signo da el resultado correcto módulo 2^{32}. Lo que **no** funciona es comparar directamente `HAL_GetTick() > tCuenta + TICK_MS`: ahí el desbordamiento sí rompe la comparación. Resta primero, compara después.

Anexo B. Por qué el error es exactamente del 9,1 %

Merece la pena hacer la cuenta, porque explica por qué un fallo de un solo carácter da un número tan concreto.

Con >=, la condición se cumple cuando han pasado 10 ms: cada centésima dura **10 ms** y todo cuadra.

Con >, la condición exige **más** de 10 ms, y como `HAL_GetTick()` avanza de milisegundo en milisegundo, el primer valor que la cumple es 11. Cada centésima dura entonces 11 ms en lugar de 10.

LA CUENTA

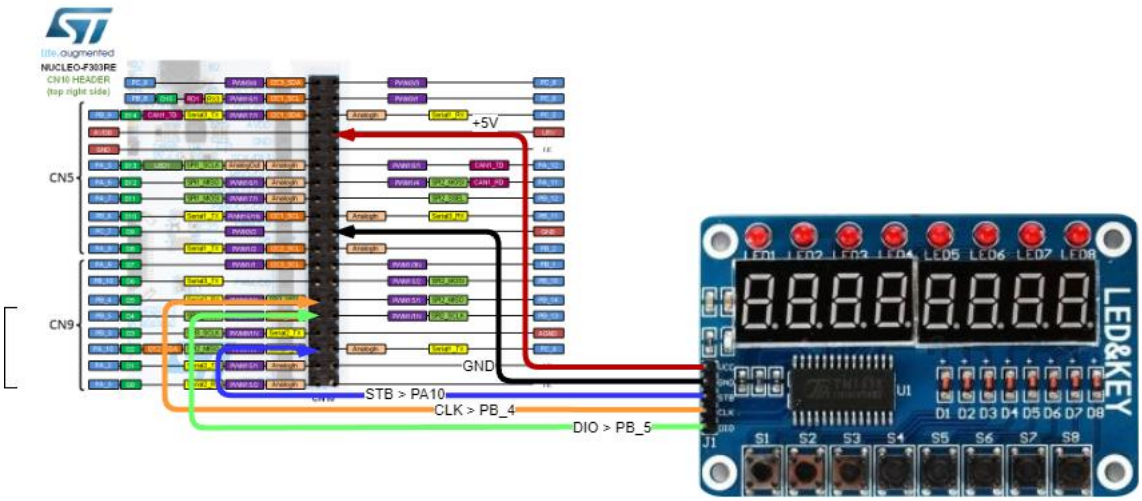
```
tiempo mostrado / tiempo real  =  10 / 11  =  0,909...
error  =  1 - 0,909  =  0,0909  =  9,1 %
```

En un minuto real el cronómetro marca 54,5 s: pierde 5,5 s. En cinco minutos pierde 27 s. Y sigue siendo un error **constante en porcentaje**, no acumulativo, que es lo que lo distingue del problema de la deriva.

Anexo C. Conexionado

El mismo de las prácticas anteriores. No hace falta cablear nada nuevo.

SEÑAL	PIN
TM1638 · STB	PA10
TM1638 · CLK	PB4
TM1638 · DIO	PB5



CONEXIÓN DEL MÓDULO LED&KEY A LA NUCLEO-F401RE.